

Major Assignment: End-to-End MLOps Pipeline Using Decision Tree Classifier and Docker

Name: Viji Benita R. Nadar

Roll No: G25AI1067

Course : MLOps

Date: 12th June 2026

GitHub Repository Link

<https://github.com/vijinadar/olivetti-mlops-major>

Docker Hub Repository Link

<https://hub.docker.com/u/vijibenita>

1. Introduction

Machine Learning Operations (MLOps) combines machine learning, DevOps, and software engineering practices to automate the development, deployment, monitoring, and maintenance of machine learning systems.

This project implements a complete MLOps workflow involving:

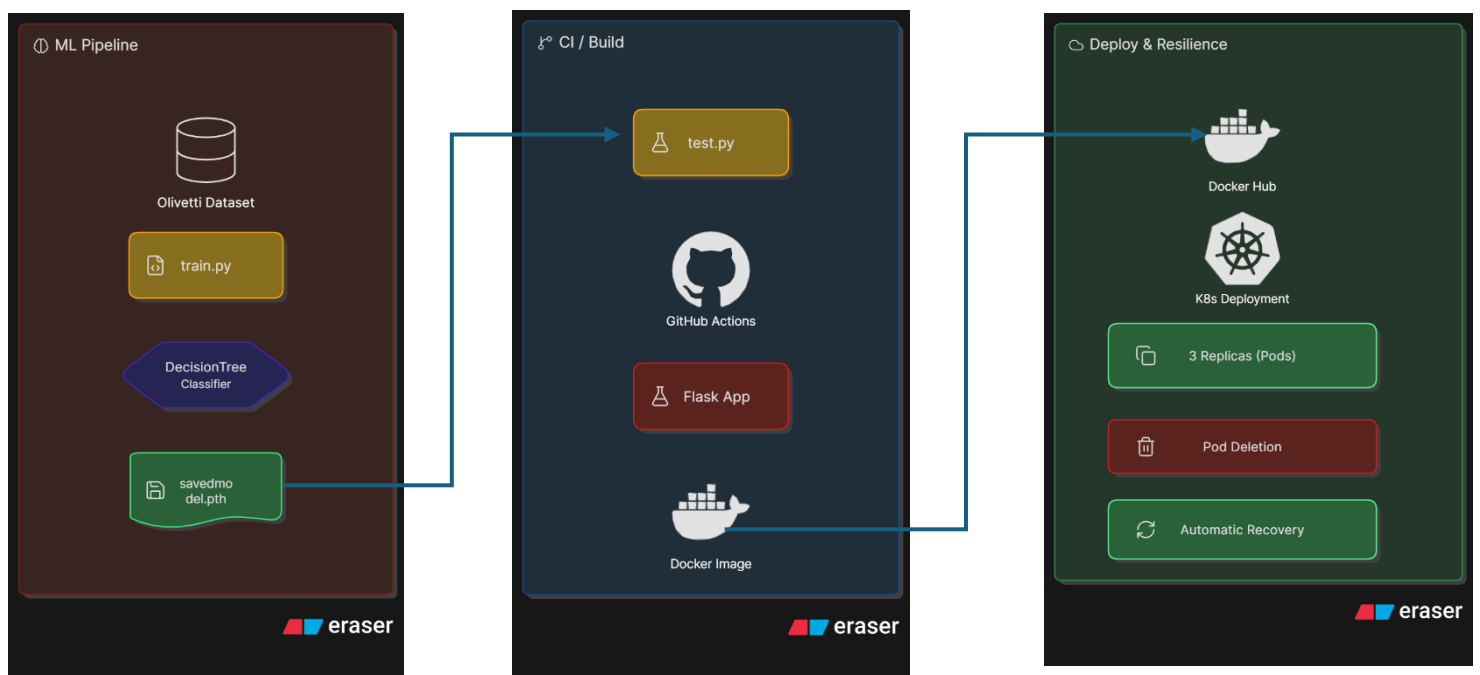
- Model development using Scikit-Learn
 - Version control using Git and GitHub
 - CI/CD automation using GitHub Actions
 - Containerization using Docker
 - Model serving using Flask
 - Deployment using Kubernetes
 - High availability using replica sets
 - Automated testing and validation
-

2. Objectives

The objectives of this assignment are:

1. Build a machine learning model using the Olivetti Faces dataset.
2. Automate training and testing using GitHub Actions.
3. Containerize the application using Docker.
4. Publish the Docker image to Docker Hub.
5. Deploy the application using Kubernetes.
6. Maintain application availability using multiple replicas.
7. Demonstrate an end-to-end MLOps workflow.

3. Project Architecture



4. Git Branching Strategy

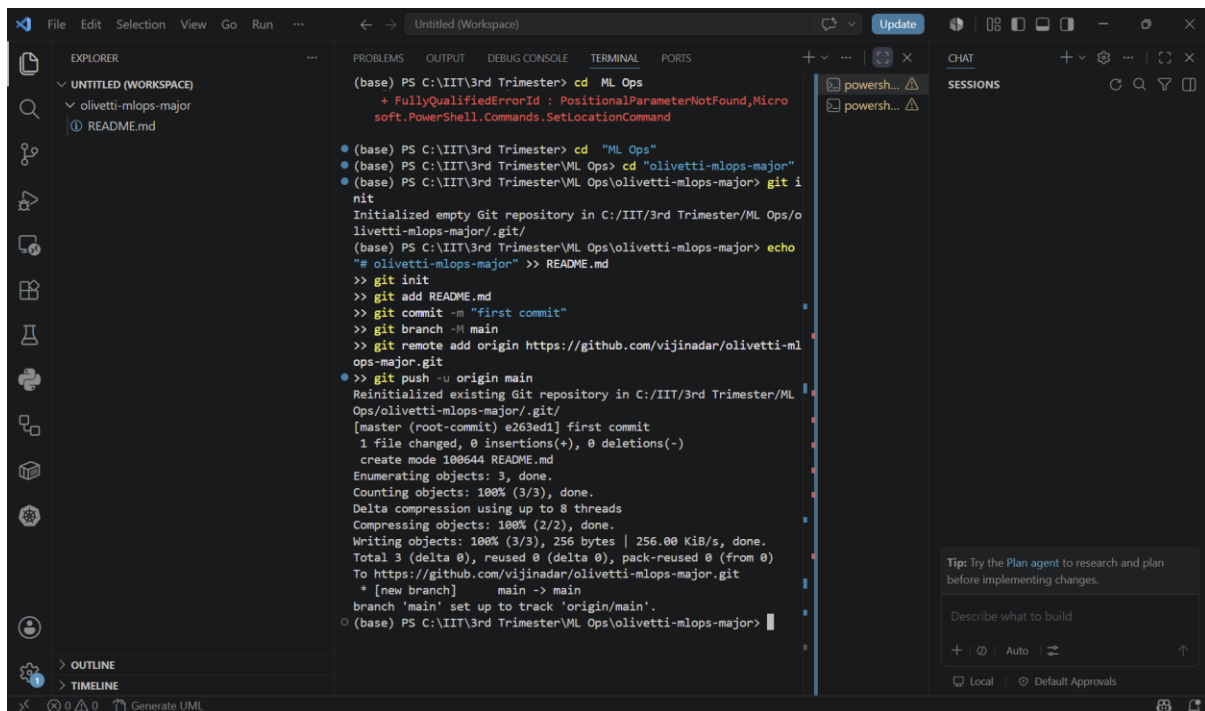
The assignment follows a three-branch workflow.

Main Branch

Initial repository setup:

- README.md
- .gitignore

Screenshot

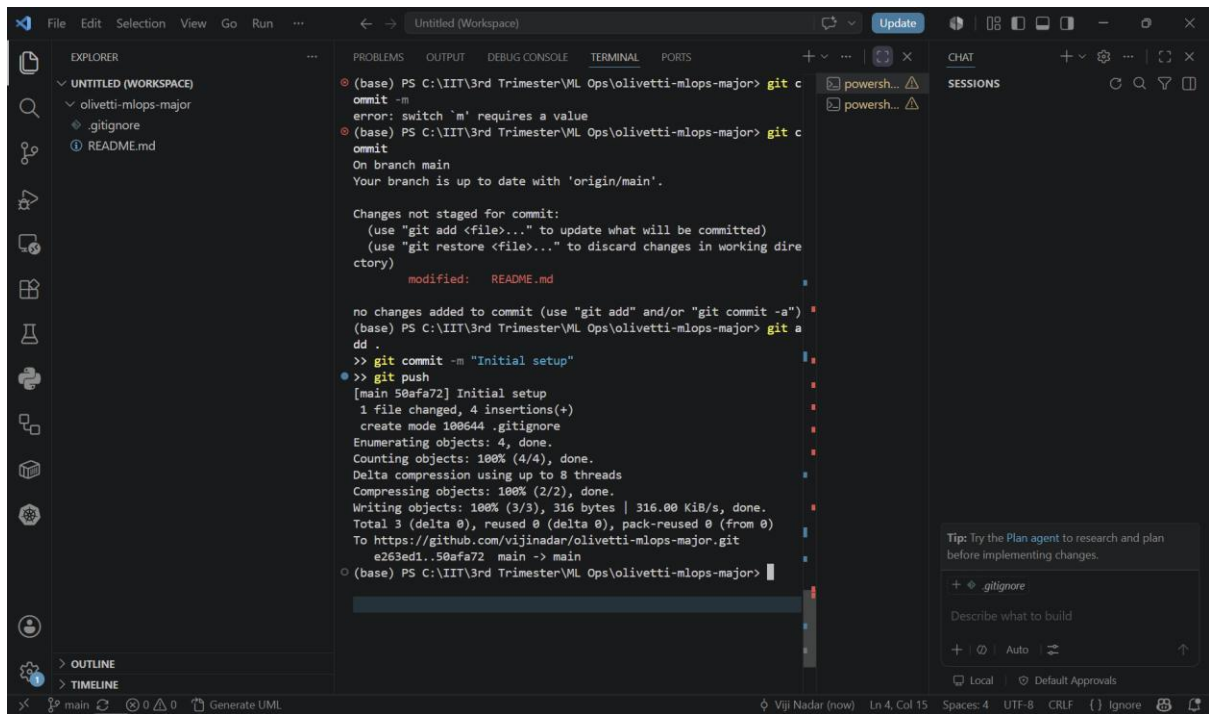


The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the following commands and output:

```
(base) PS C:\IIT\3rd Trimester> cd ML Ops
+ FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

• (base) PS C:\IIT\3rd Trimester> cd "ML Ops"
• (base) PS C:\IIT\3rd Trimester\ML Ops> cd "olivetti-mlops-major"
• (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major> git init
Initialized empty Git repository in C:/IIT/3rd Trimester/ML Ops/olivetti-mlops-major/.git/
(base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major> echo "# olivetti-mlops-major" >> README.md
>> git init
>> git add README.md
>> git commit -m "first commit"
>> git branch -M main
>> git remote add origin https://github.com/vijnadar/olivetti-mlops-major.git
• >> git push -u origin main
Reinitialized existing Git repository in C:/IIT/3rd Trimester/ML Ops/olivetti-mlops-major/.git/
[master (root-commit) e263ed1] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 256 bytes | 256.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/vijnadar/olivetti-mlops-major.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
(base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major>
```

The Explorer view on the left shows the file structure with 'olivetti-mlops-major' and 'README.md'. The Output view on the right shows the chat interface with a tip to try the Plan agent.

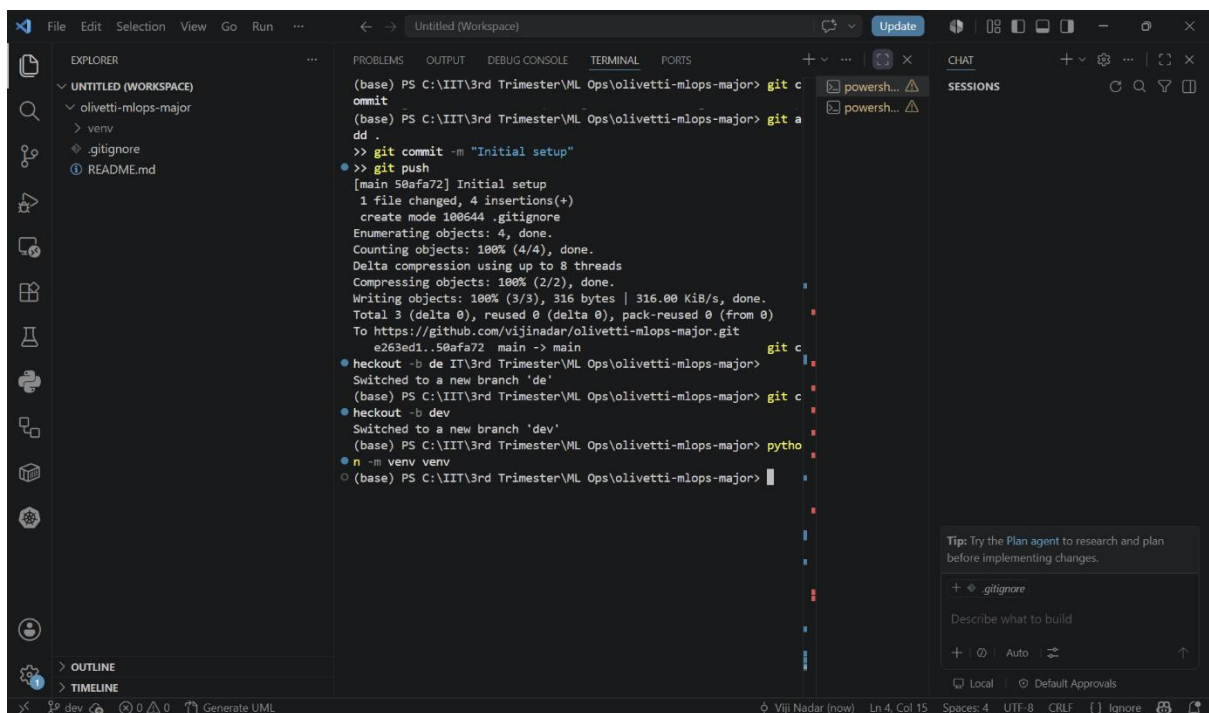


Dev Branch

Created for machine learning development.

git checkout -b dev

Screenshot

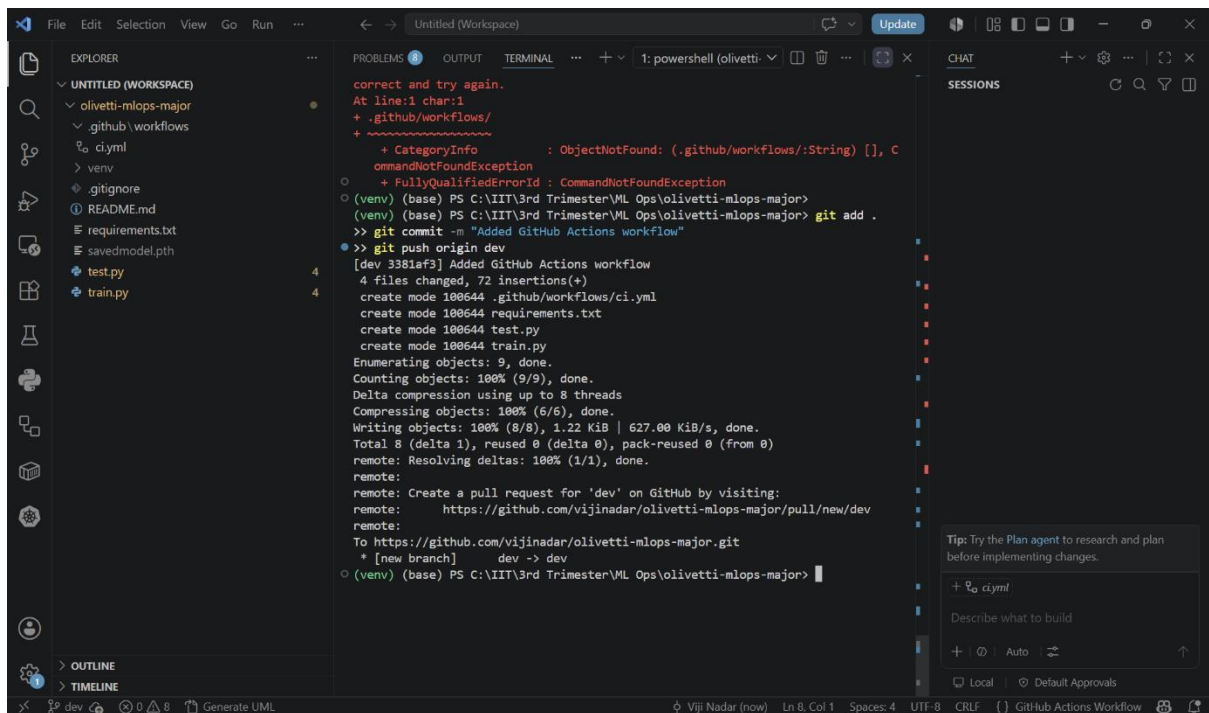


docker_cicd Branch

Created from dev branch.

git checkout -b docker_cicd

Screenshot



5. Dataset Description

Dataset used:

Olivetti Faces Dataset

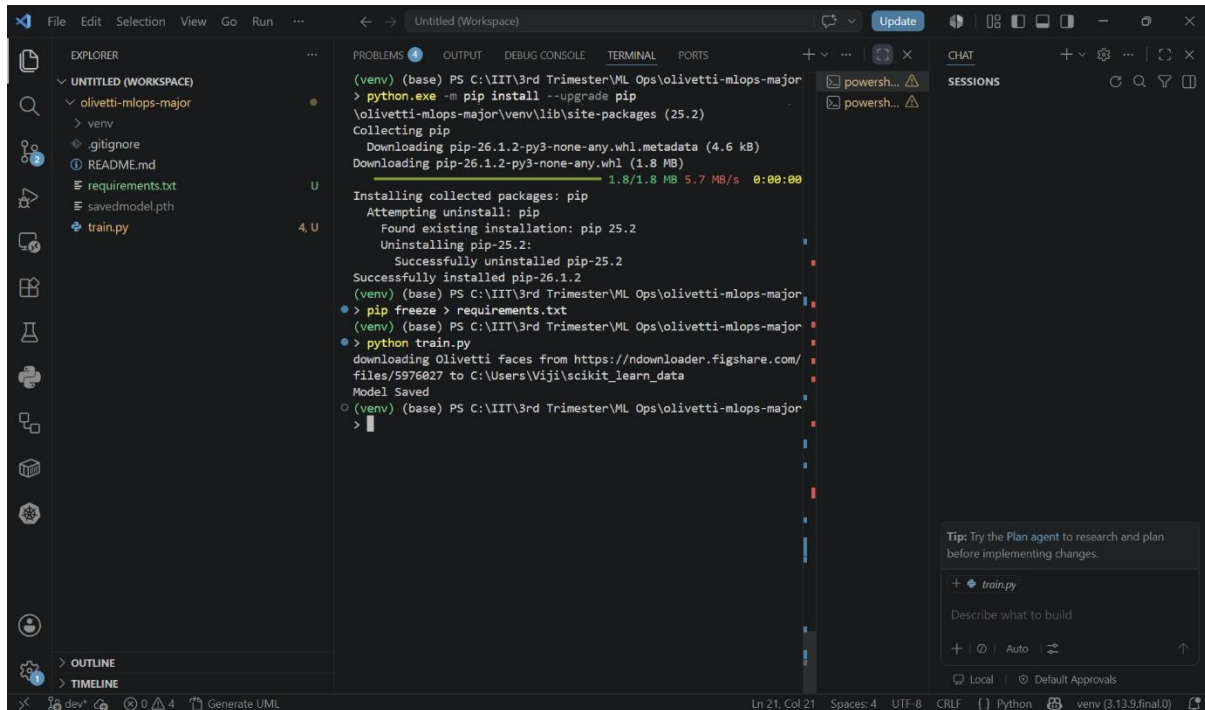
Features:

- 400 face images
- 40 individuals
- 10 images per individual
- Image size: 64 × 64 pixels

Loaded using:

```
from sklearn.datasets import fetch_olivetti_faces
```

Screenshot



6. Model Development

Training Script (train.py)

Steps performed:

1. Load dataset
2. Split dataset into:
 - 70% Training
 - 30% Testing
3. Train Decision Tree Classifier
4. Save model using Joblib

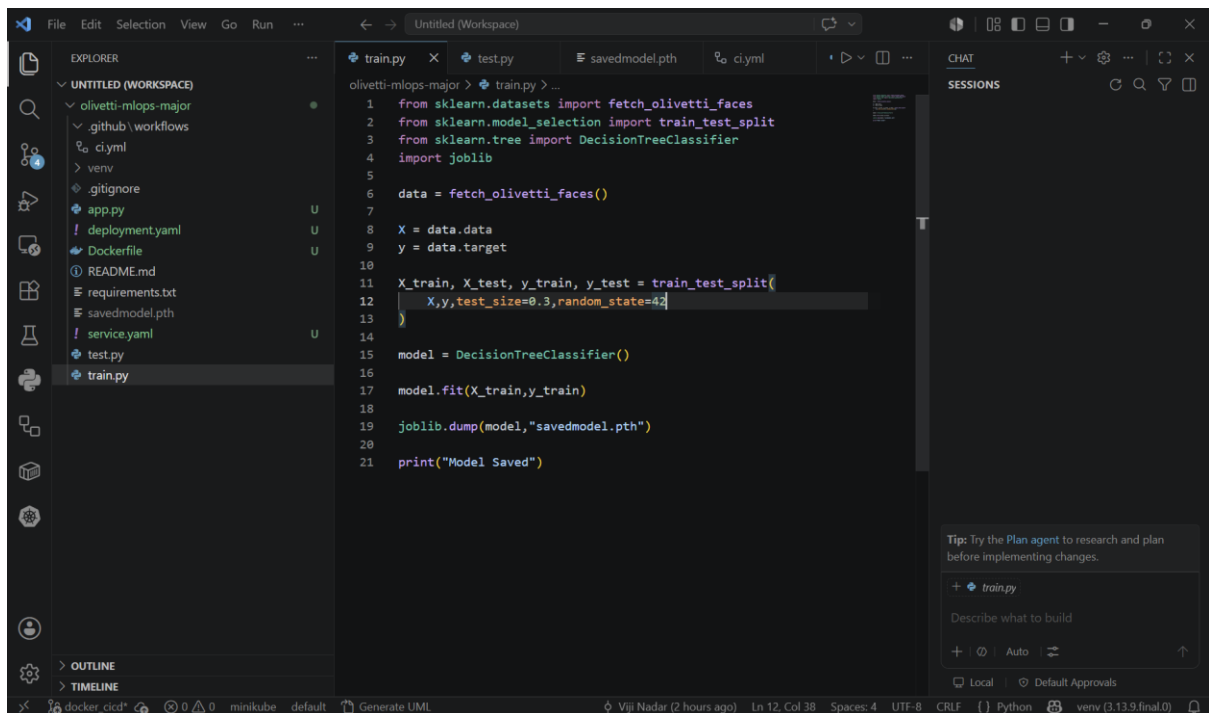
Example:

```
clf = DecisionTreeClassifier()
```

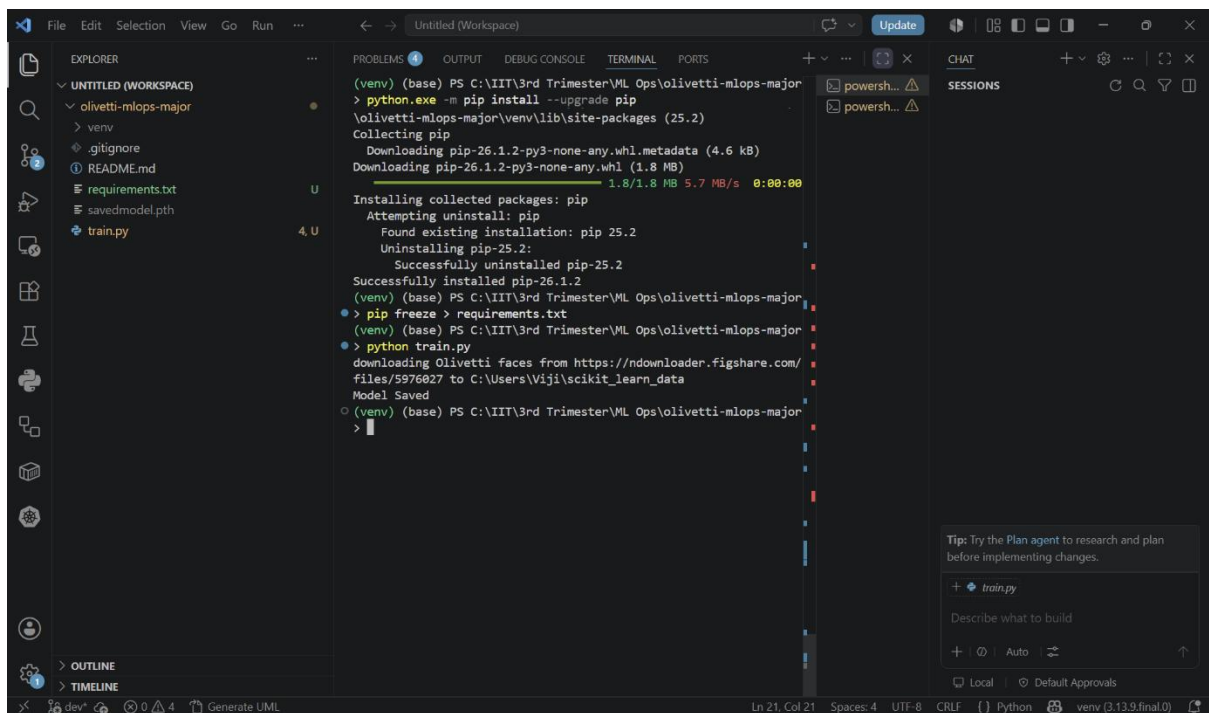
```
clf.fit(X_train, y_train)
```

```
joblib.dump(clf, "savedmodel.pth")
```

Screenshot



Output Screenshot



7. Model Testing

test.py

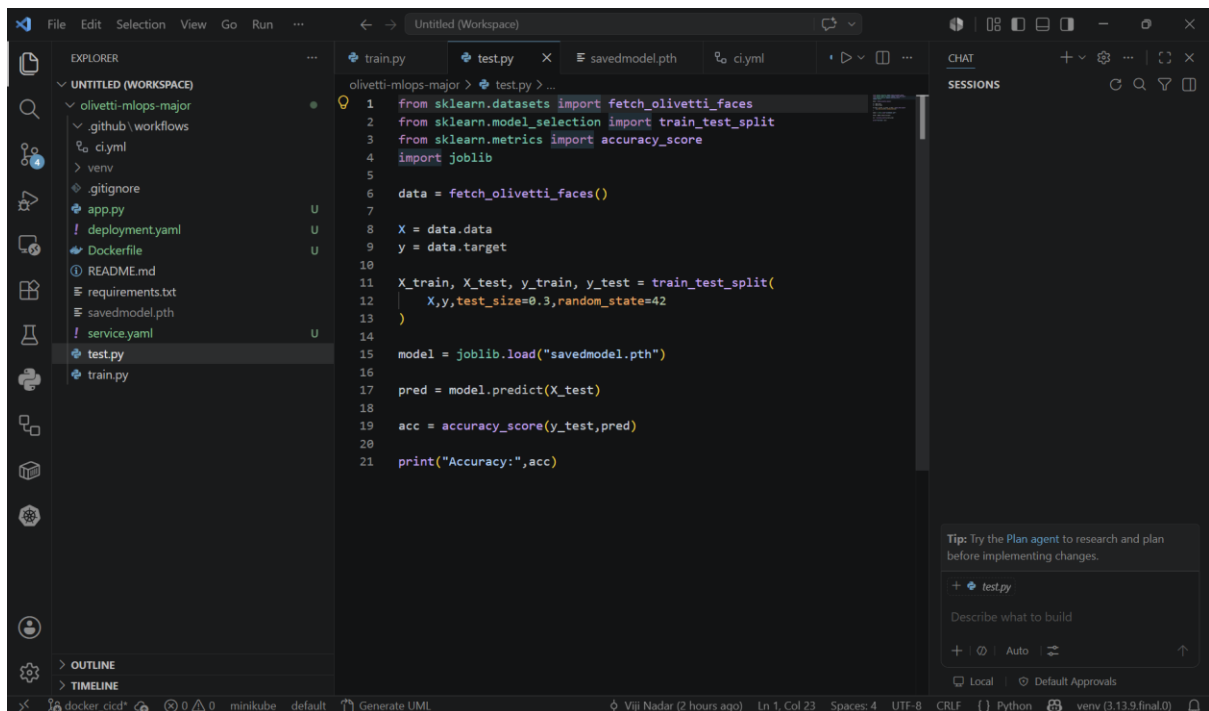
Steps:

1. Load saved model
2. Load test dataset
3. Predict classes
4. Compute accuracy

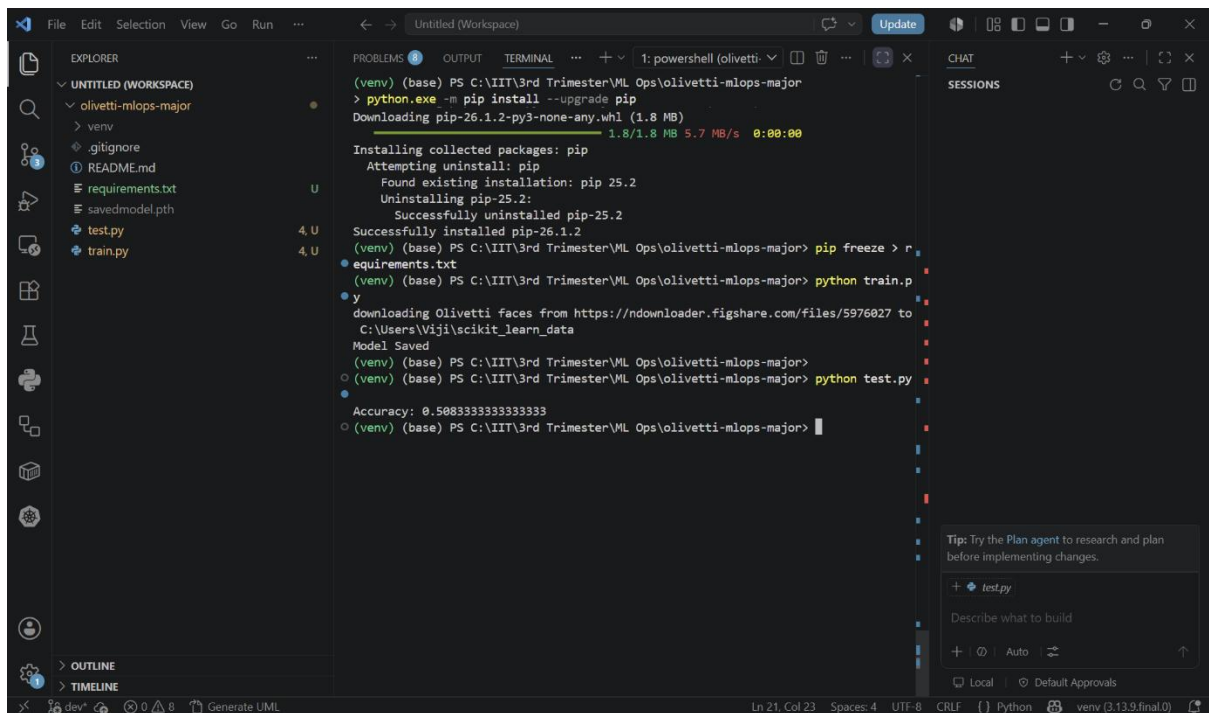
```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Screenshot



Output Screenshot



```
(venv) (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major
> python.exe -m pip install --upgrade pip
Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)
1.8/1.8 MB 5.7 MB/s 0:00:00

Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
  Successfully installed pip-26.1.2
(venv) (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major> pip freeze > r
equirements.txt
(venv) (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major> python train.p
y
downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to
C:\Users\Viiji\scikit_learn_data
Model Saved
(venv) (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major>
(venv) (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major> python test.py
Accuracy: 0.5083333333333333
(venv) (base) PS C:\IIT\3rd Trimester\ML Ops\olivetti-mlops-major>
```

8. GitHub Actions CI/CD Pipeline

Location:

.github/workflows/ci.yml

Purpose:

- Install dependencies
- Run train.py
- Generate model
- Execute test.py
- Display accuracy

Workflow Steps:

Step 1

Checkout repository

- uses: actions/checkout@v4

Step 2

Setup Python

- uses: actions/setup-python@v5

Step 3

Install requirements

pip install -r requirements.txt

Step 4

Train model

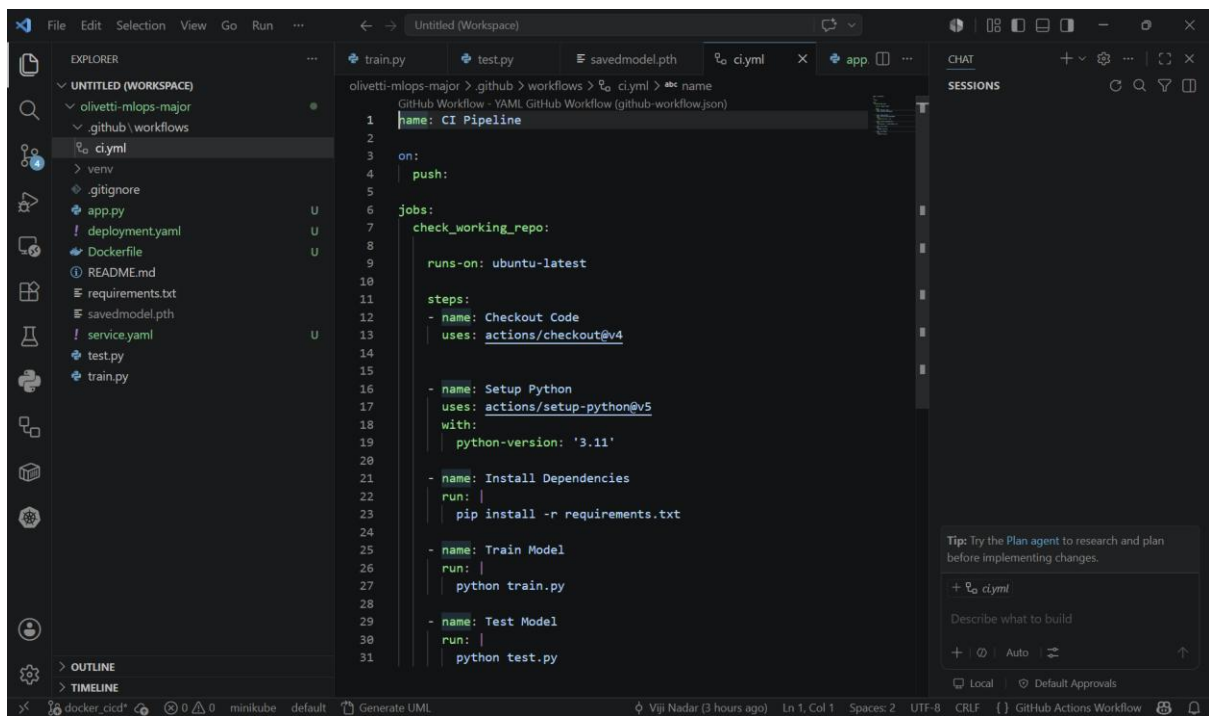
python train.py

Step 5

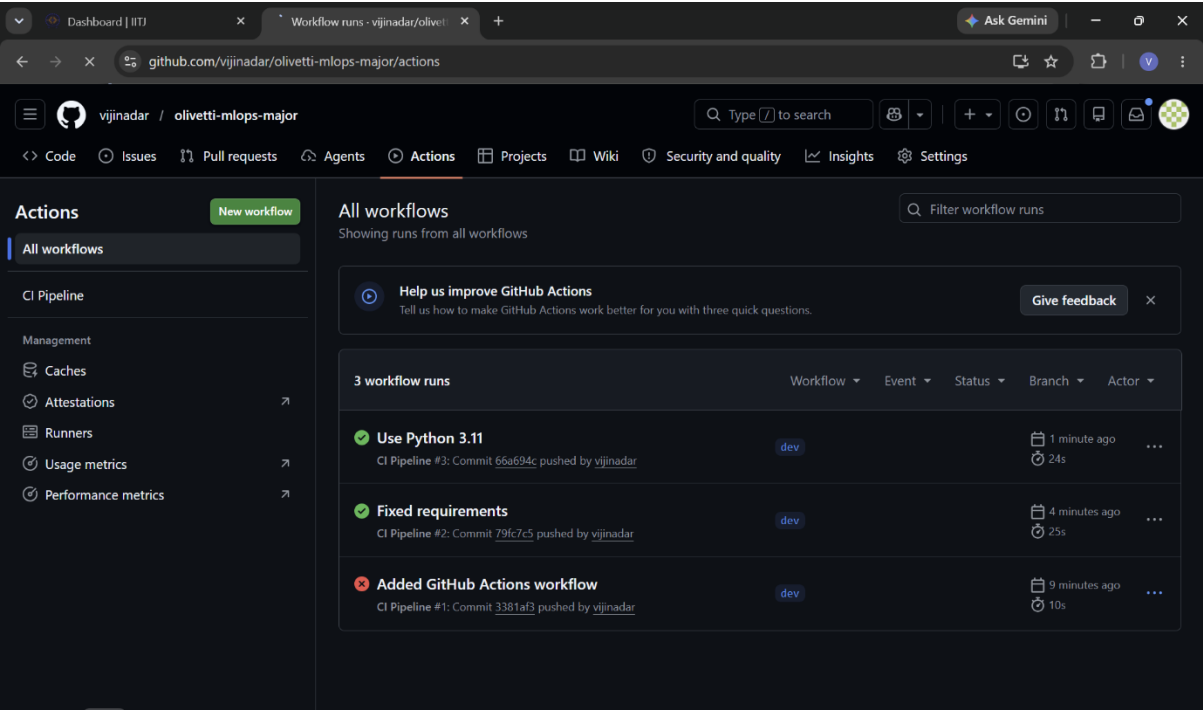
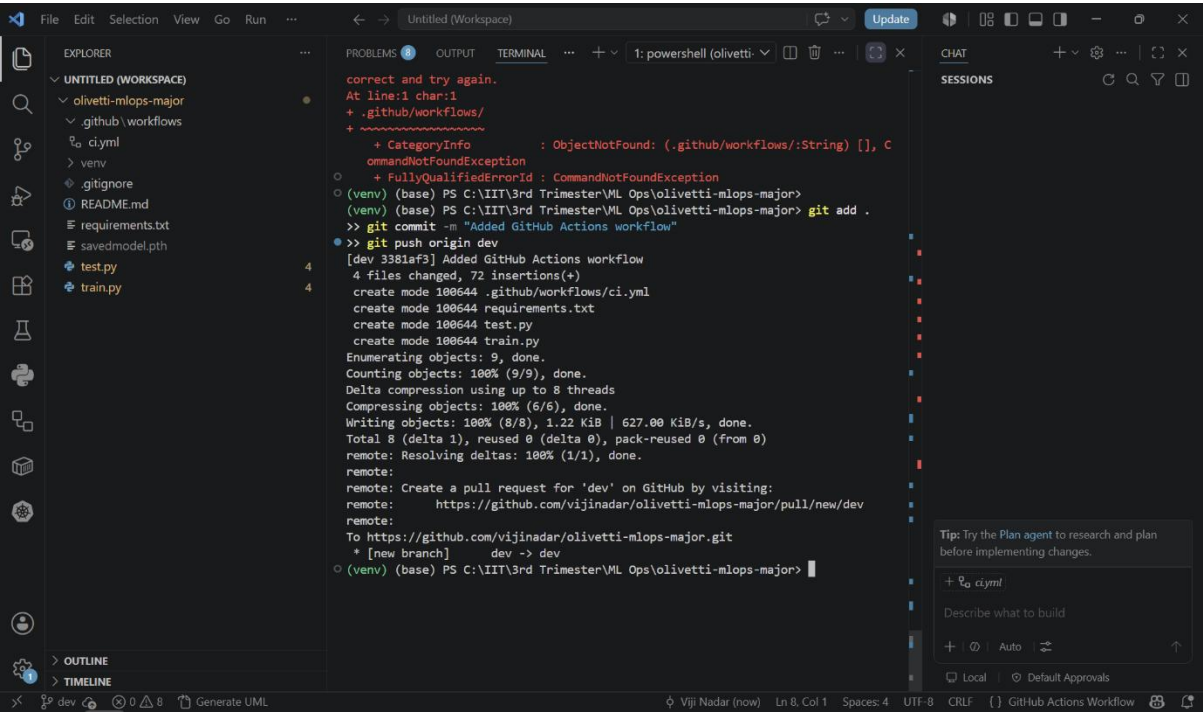
Test model

python test.py

Screenshot



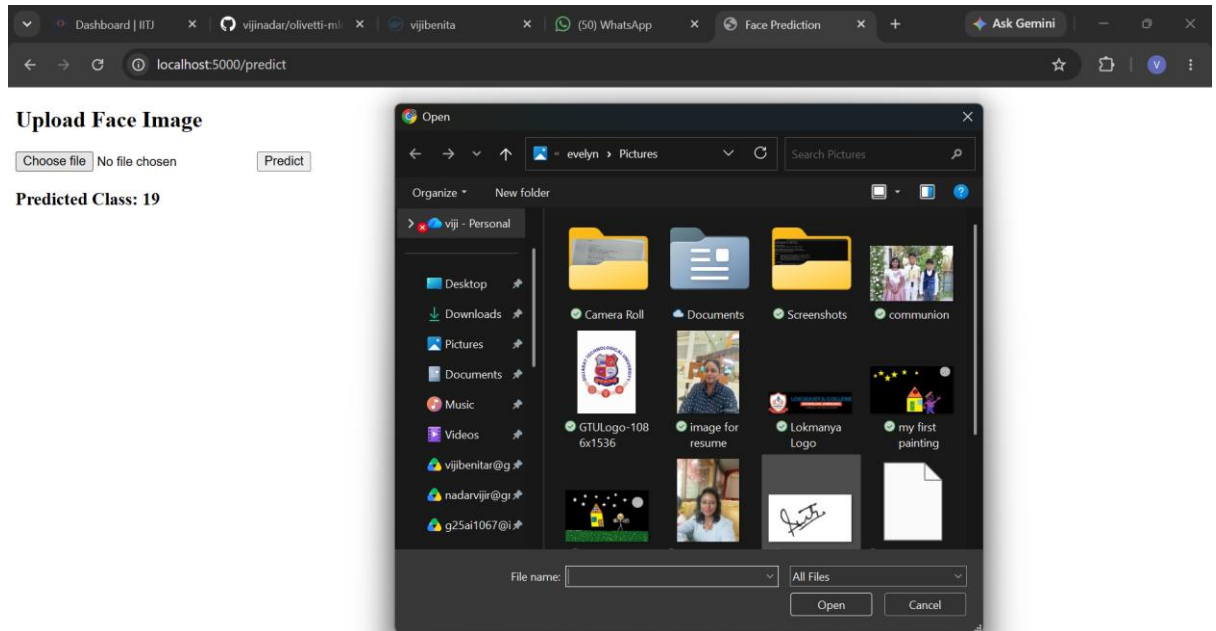
Screenshot



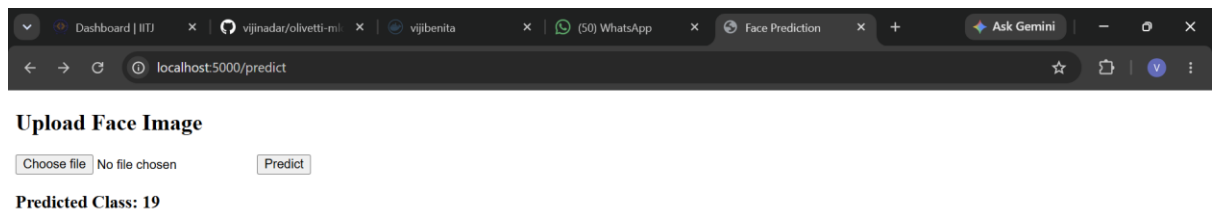
9. Flask Web Application

The Flask application allows users to:

- Upload face images



- Run predictions
- Display predicted class



Project structure:

app.py

templates/

index.html

savedmodel.pth

Screenshot

The screenshot shows the VS Code editor interface. The Explorer panel on the left displays the project structure for 'olivetti-mlops-major', including files like .github/workflows, ci.yml, templates/index.html, .venv, .gitignore, app.py, deployment.yml, Dockerfile, README.md, requirements.txt, savedmodel.pth, service.yml, test.py, and train.py. The main editor area shows the code for app.py, which is a Flask application. It imports Flask, render_template, request, joblib, numpy, and PIL Image. The app is configured with Flask(__name__) and has two routes: a home route that renders index.html, and a predict route that takes an image file, processes it with a pre-trained model (loaded from savedmodel.pth), and returns the prediction. The predict route uses request.files to get the image, converts it to a numpy array, and then uses the model's predict method. The app is run on host '0.0.0.0' and port 5000.

```
1 from flask import Flask, render_template, request
2 import joblib
3 import numpy as np
4 from PIL import Image
5
6 app = Flask(__name__)
7
8 model = joblib.load("savedmodel.pth")
9
10 @app.route("/")
11 def home():
12     return render_template("index.html")
13
14 @app.route("/predict", methods=["POST"])
15 def predict():
16     file = request.files["image"]
17
18     img = Image.open(file).convert("L")
19     img = img.resize((64, 64))
20
21     img_array = np.array(img).flatten().reshape(1, -1) / 255.0
22
23     prediction = model.predict(img_array)[0]
24
25     return render_template(
26         "index.html",
27         prediction=prediction
28     )
29
30 if __name__ == "__main__":
31     app.run(host="0.0.0.0", port=5000)
```

The screenshot shows the VS Code editor interface with the terminal panel open. The terminal displays the output of running the app.py script. It shows the server starting on http://192.168.1.6:5000. A GET request is received, followed by a POST request to /predict. The server returns a 200 status code for both. However, there is a traceback error in the app.py file, specifically in the predict function, where an UnidentifiedImageError is raised because the image file cannot be identified. The error message is: 'PIL.UnidentifiedImageError: cannot identify image file <FileStorage: '' ('application/octet-stream')>'. The terminal output is as follows:

```
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major> python app.py
* Running on http://192.168.1.6:5000
Press CTRL+C to quit
127.0.0.1 - - [12/Jun/2026 18:59:36] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 18:59:53] "POST /predict HTTP/1.1" 200 -
[2026-06-12 19:01:41,954] ERROR in app: Exception on /predict [POST]
Traceback (most recent call last):
  File "C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
  File "C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major\venv\Lib\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
  File "C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major\venv\Lib\site-packages\flask\app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
  File "C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major\venv\Lib\site-packages\flask\app.py", line 902, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
  File "C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major\app.py", line 18, in predict
    img = Image.open(file).convert("L")
  File "C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major\venv\Lib\site-packages\PIL\image.py", line 3711, in open
    raise UnidentifiedImageError(msg)
PIL.UnidentifiedImageError: cannot identify image file <FileStorage: '' ('application/octet-stream')>
127.0.0.1 - - [12/Jun/2026 19:01:41] "POST /predict HTTP/1.1" 500 -
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major>
```

10. Dockerization

Dockerfile

Purpose:

- Create portable environment
- Run Flask application inside container

Example:

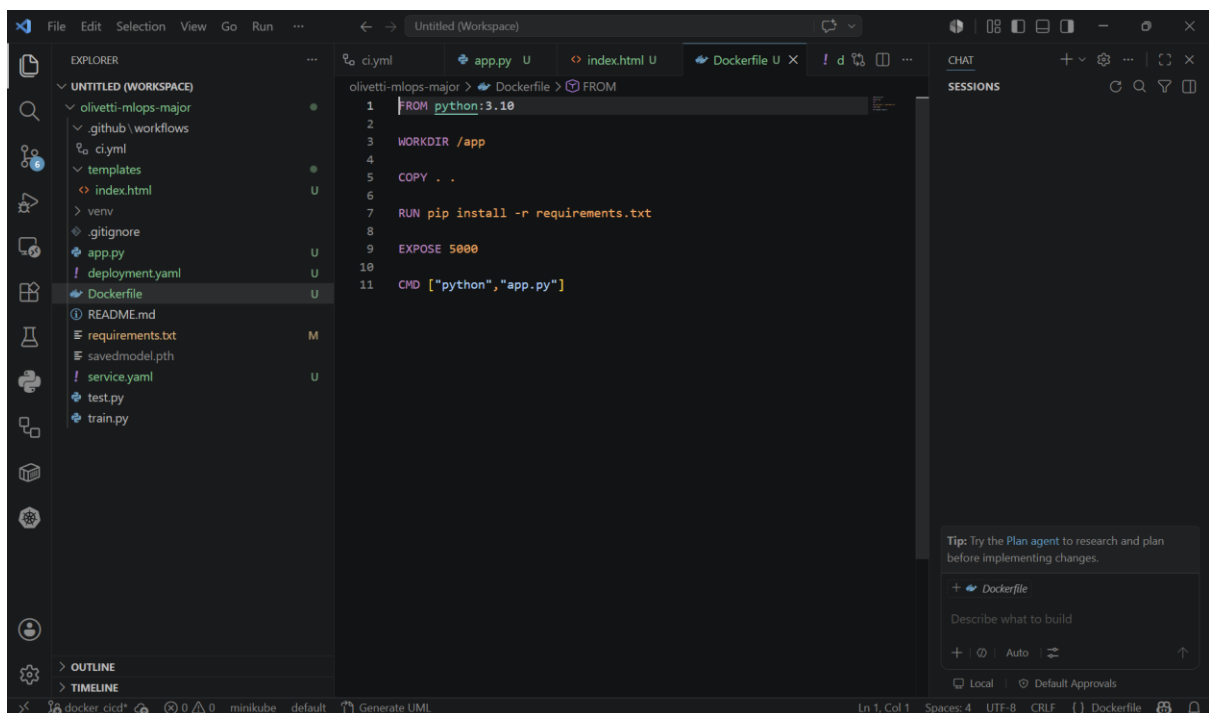
FROM python:3.10

WORKDIR /app

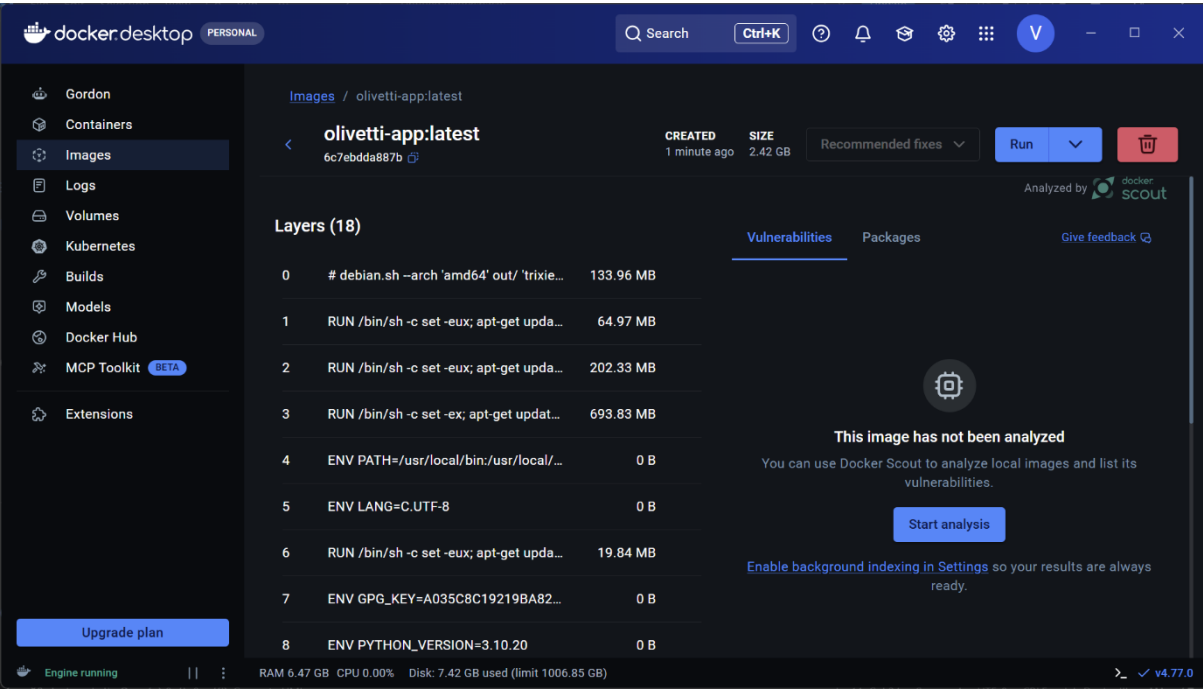
COPY . .

RUN pip install -r requirements.txt

CMD ["python","app.py"]



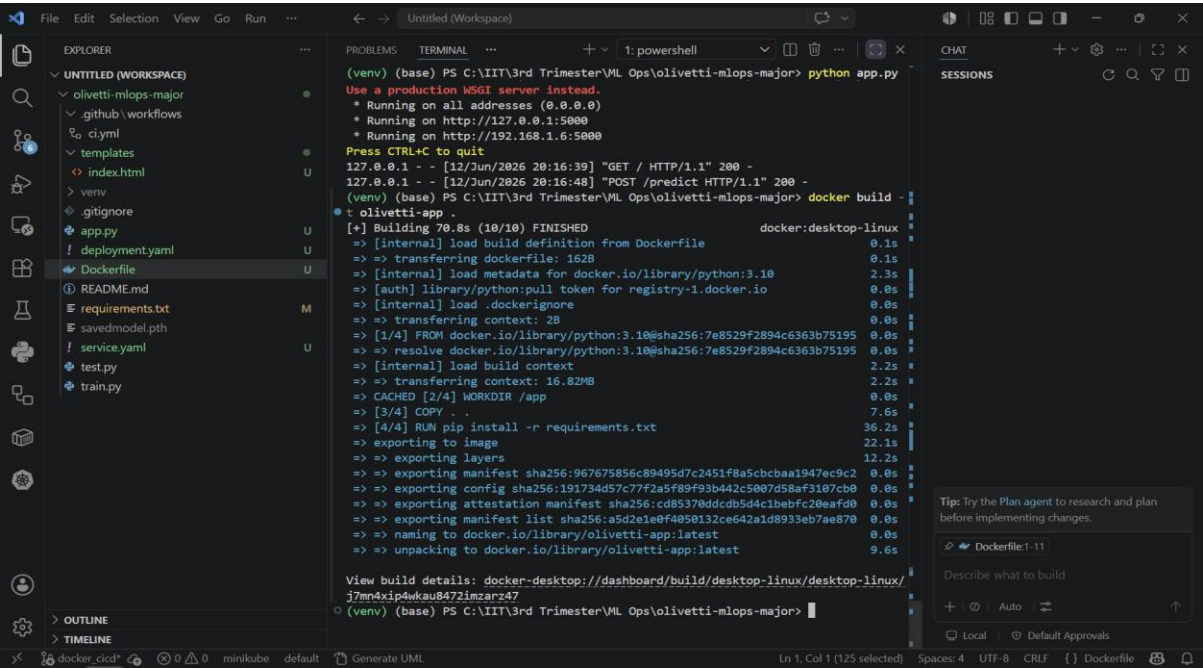
Screenshot



Building Docker Image

docker build -t olivetti-flask-app .

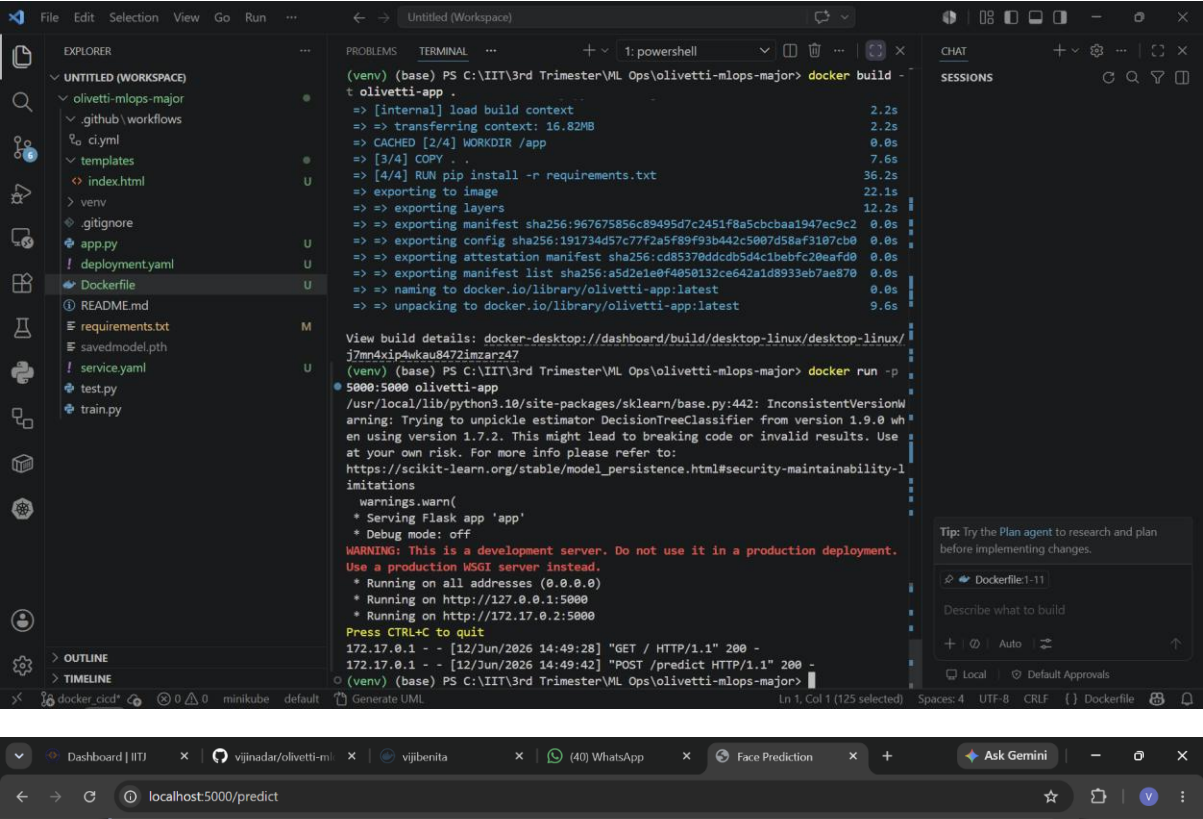
Screenshot

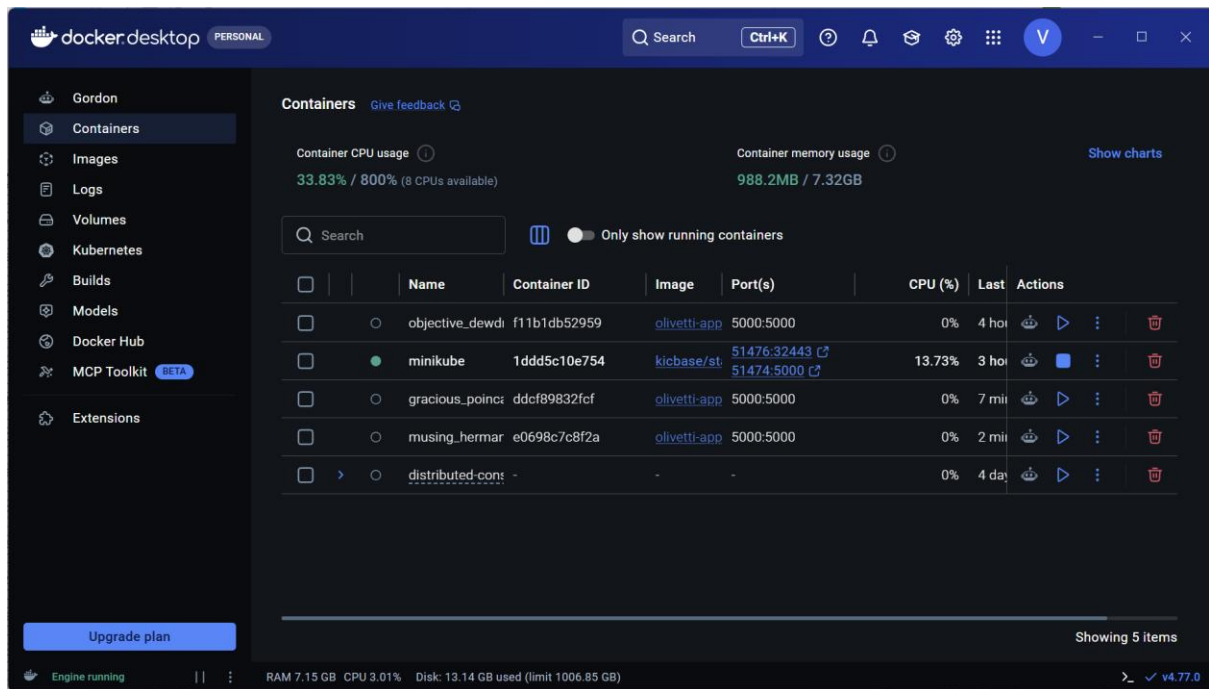


Running Docker Container

docker run -p 5000:5000 olivetti-flask-app

Screenshot





11. Docker Hub Integration

Login:

```
docker login
```

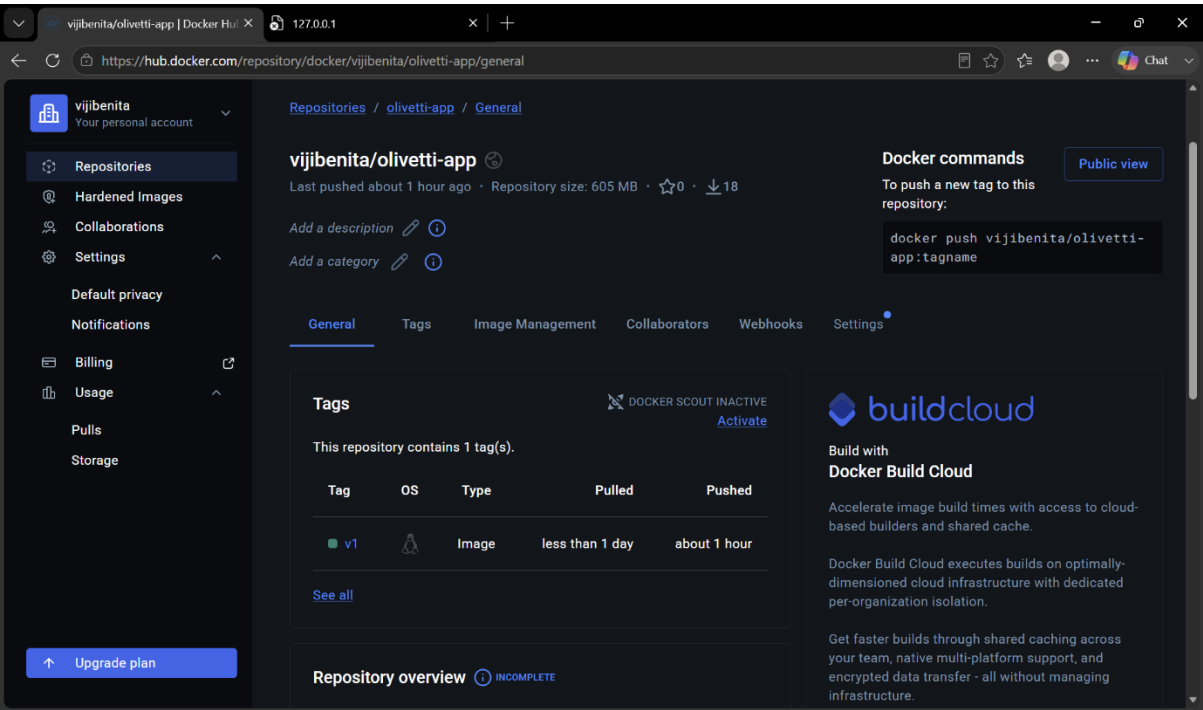
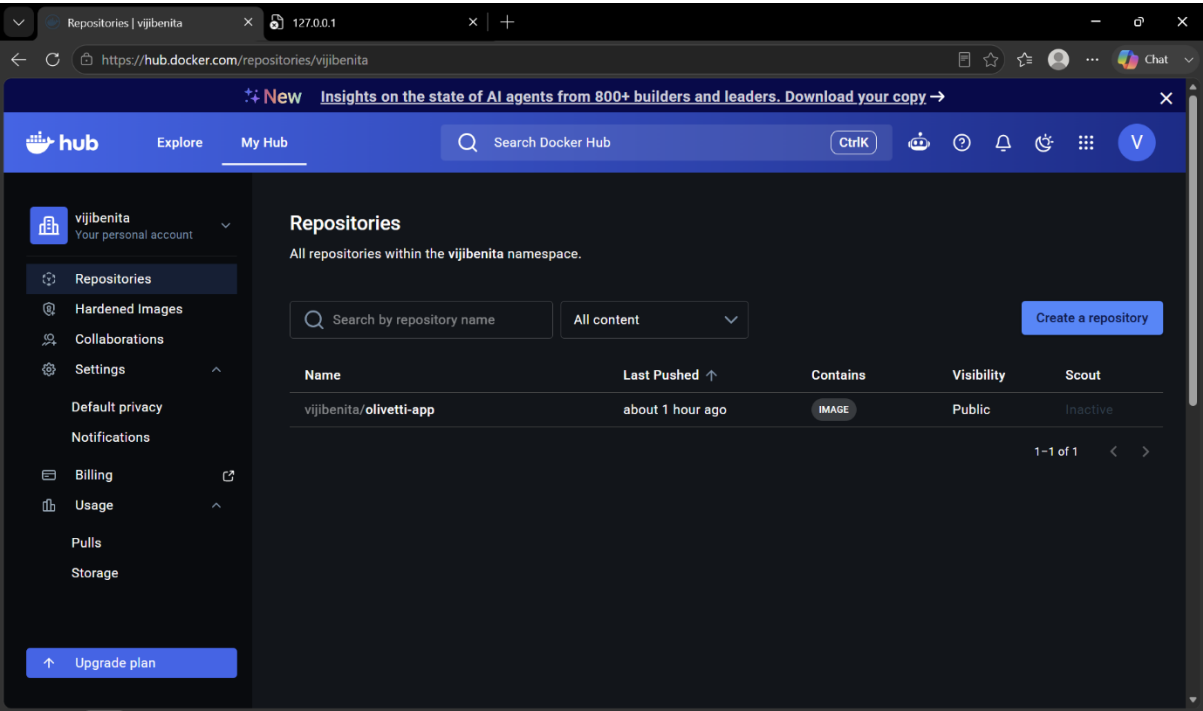
Tag Image:

```
docker tag olivetti-flask-app username/olivetti-flask-app:latest
```

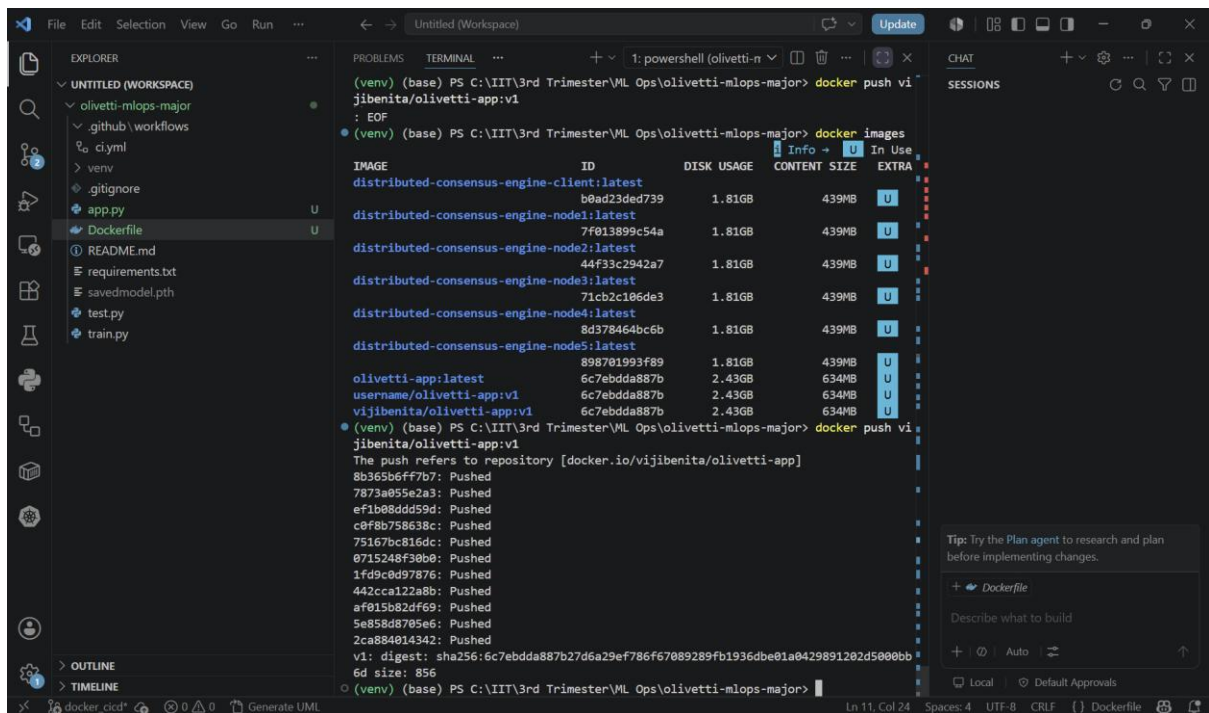
Push Image:

```
docker push username/olivetti-flask-app:latest
```

Screenshot



Screenshot



12. Kubernetes Deployment

Deployment file:

apiVersion: apps/v1

kind: Deployment

metadata:

name: olivetti-deployment

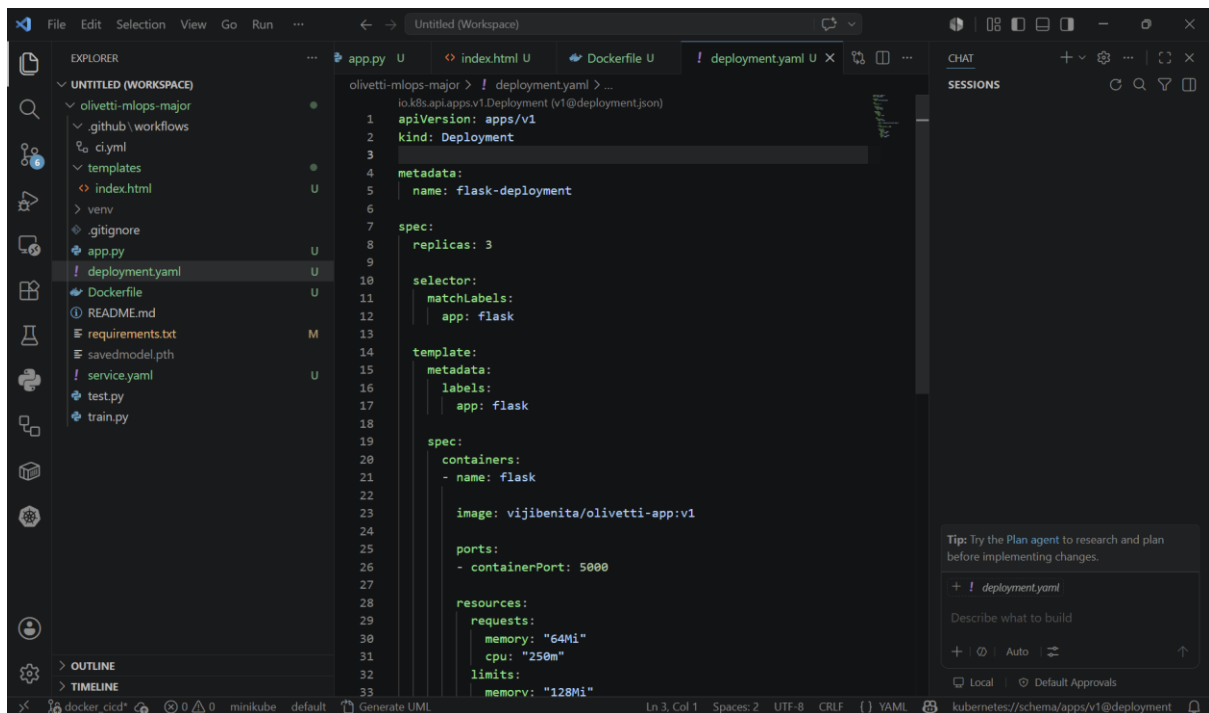
spec:

replicas: 3

Purpose:

- Maintain 3 running replicas
- Automatic recovery

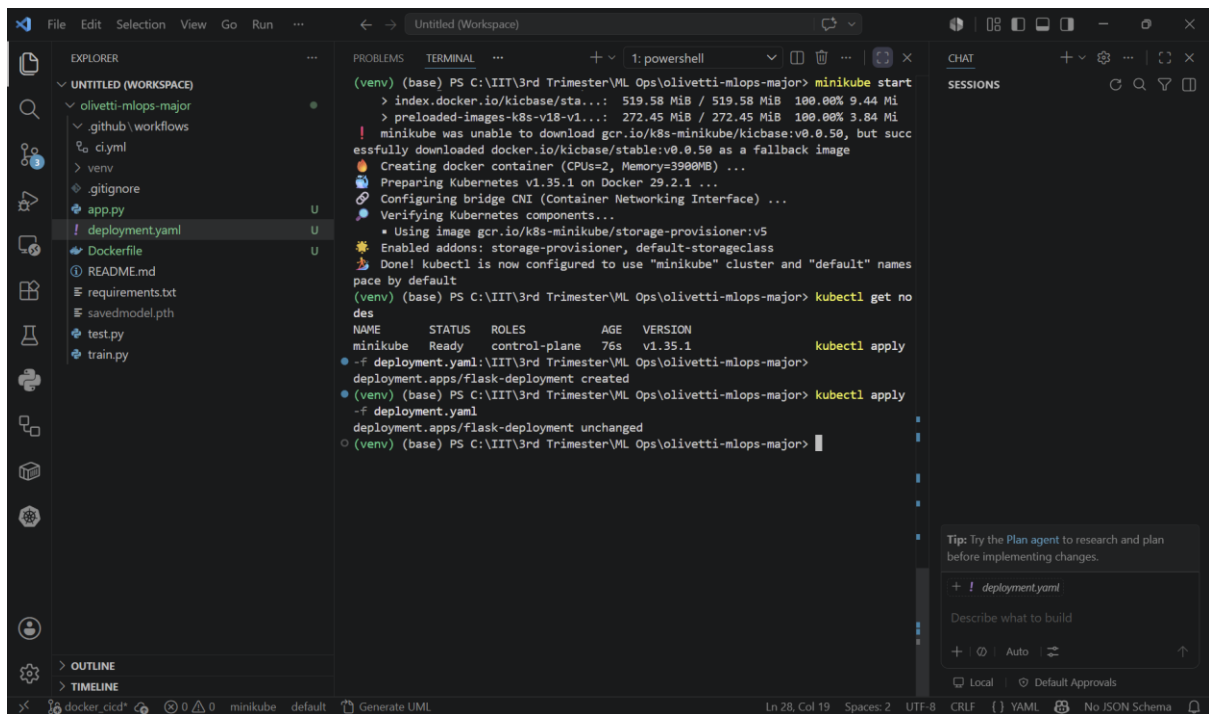
Screenshot



Deployment Command

kubectl apply -f deployment.yaml

Screenshot



13. NodePort Service

Service file:

kind: Service

type: NodePort

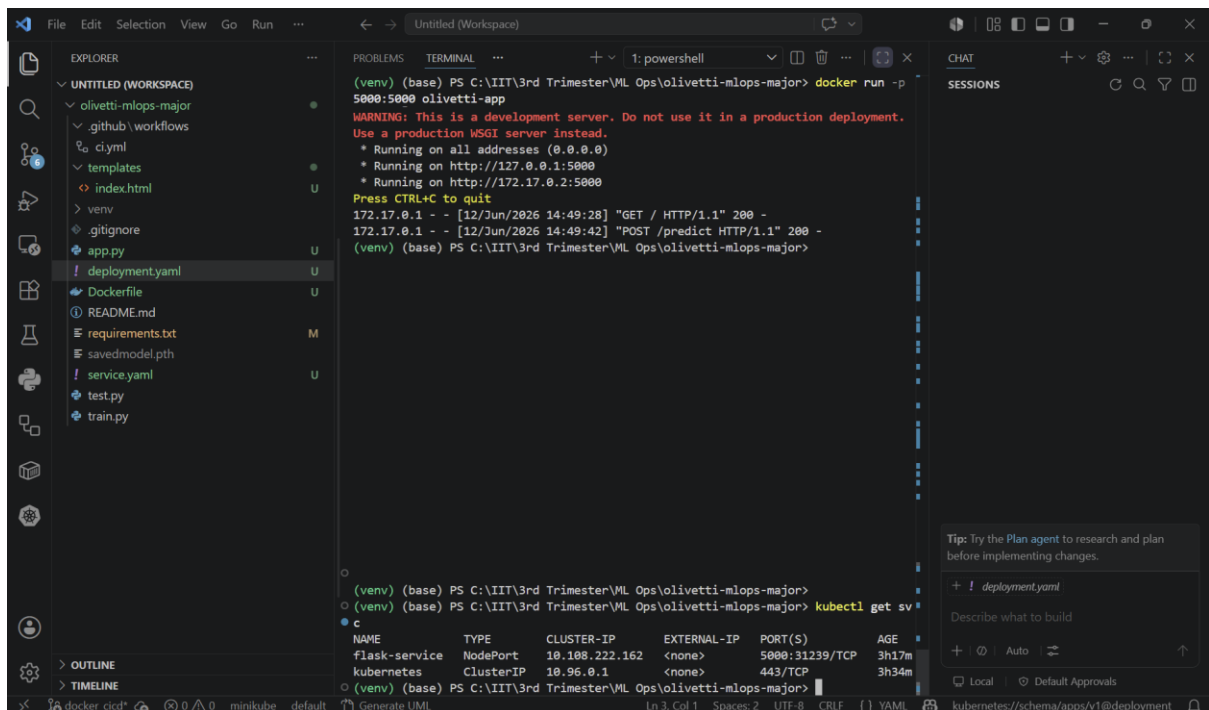
Apply:

```
kubectl apply -f service.yaml
```

Check:

```
kubectl get svc
```

Screenshot



14. Accessing the Application

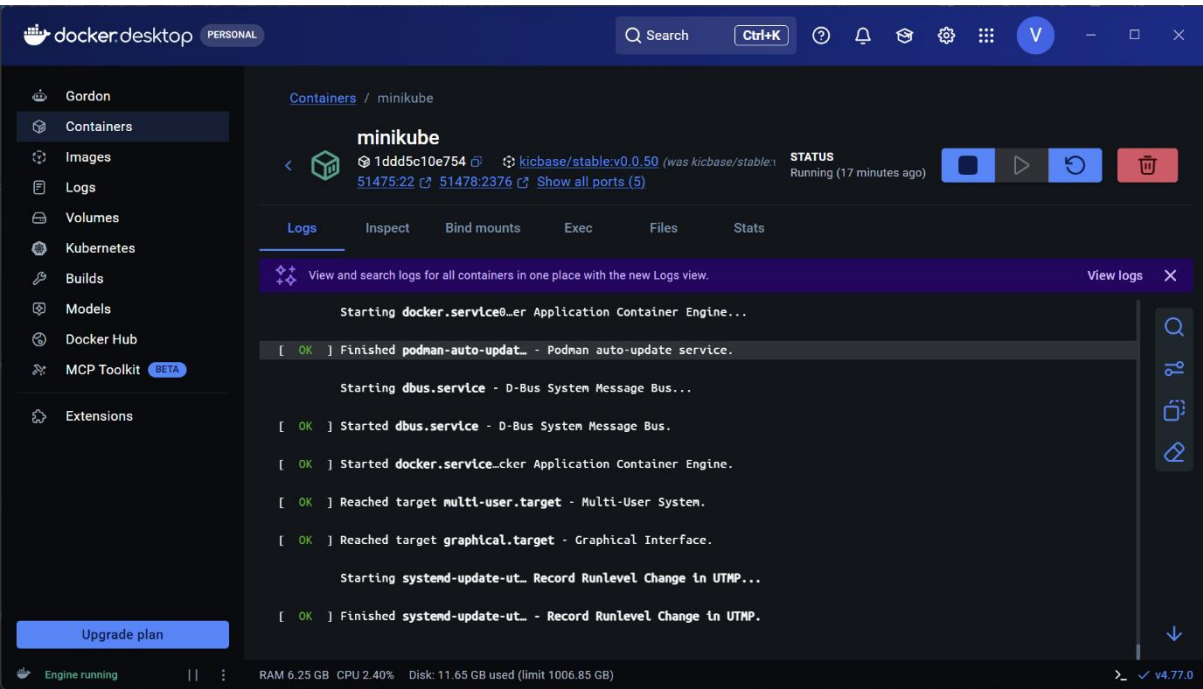
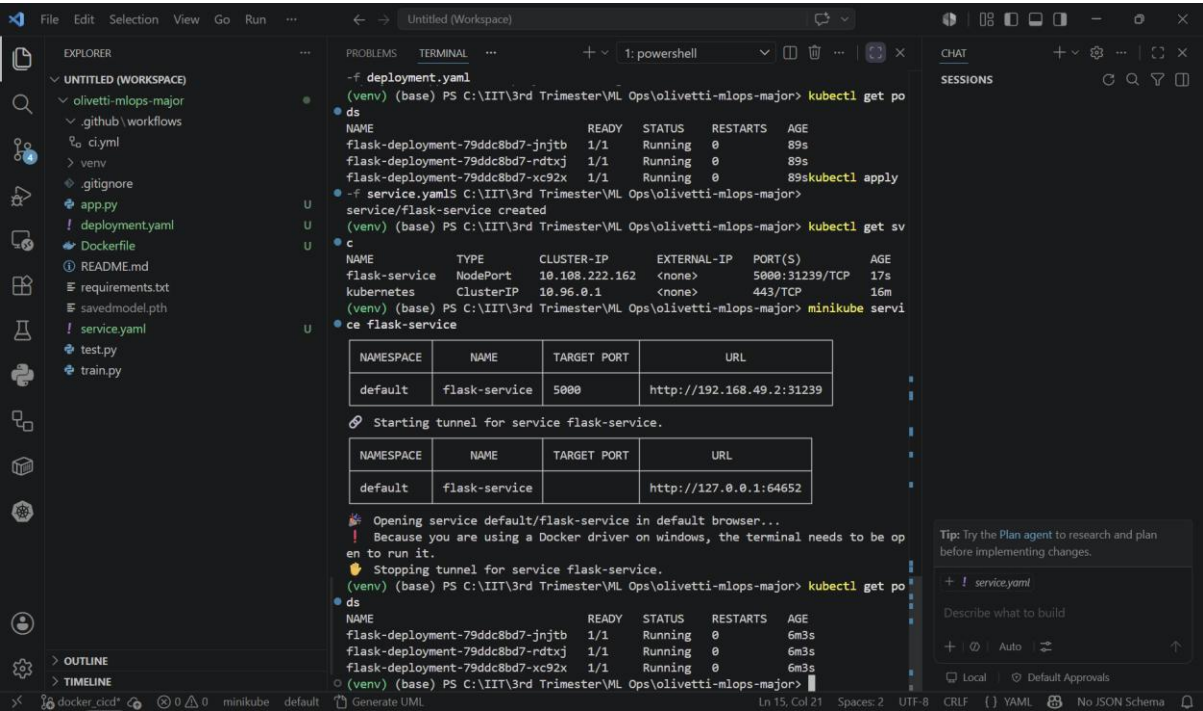
Obtain URL:

minikube service flask-service

or

```
kubectl get svc
```

Screenshot



15. Replica Verification

Check running pods:

kubectl get pods

Expected:

3/3 Running

Screenshot

The screenshot shows a VS Code terminal window with a PowerShell prompt. The user has executed several commands to verify a Kubernetes deployment. The output shows three pods in a 'Running' state. A service named 'flask-service' is also shown, with a target port of 5000 and a URL of http://192.168.49.2:31239. The terminal also shows a warning about the Docker driver on Windows and a tip to try the Plan agent.

```
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major> kubectl get po
ds
NAME                                READY   STATUS    RESTARTS   AGE
flask-deployment-79ddc8bd7-jnjb     1/1     Running   0           89s
flask-deployment-79ddc8bd7-rdtxj     1/1     Running   0           89s
flask-deployment-79ddc8bd7-xc92x     1/1     Running   0           89s
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major> kubectl apply
-f service.yaml
service/flask-service created
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major> kubectl get sv
c
NAME    TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
flask-service  NodePort    10.108.222.162  <none>        5000:31239/TCP  17s
kubernetes  ClusterIP   10.96.0.1      <none>        443/TCP      16m
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major> minikube servi
ce flask-service
Starting tunnel for service flask-service.
NAME    TARGET PORT   URL
default flask-service  5000         http://127.0.0.1:64652
Opening service default/flask-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be op
en to run it.
Stopping tunnel for service flask-service.
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major> kubectl get po
ds
NAME                                READY   STATUS    RESTARTS   AGE
flask-deployment-79ddc8bd7-jnjb     1/1     Running   0           6m3s
flask-deployment-79ddc8bd7-rdtxj     1/1     Running   0           6m3s
flask-deployment-79ddc8bd7-xc92x     1/1     Running   0           6m3s
(venv) (base) PS C:\IIT\3rd Trimester\VL Ops\olivetti-mlops-major>
```

16. Fault Tolerance Demonstration

Delete one pod:

kubectl delete pod pod-name

Observe:

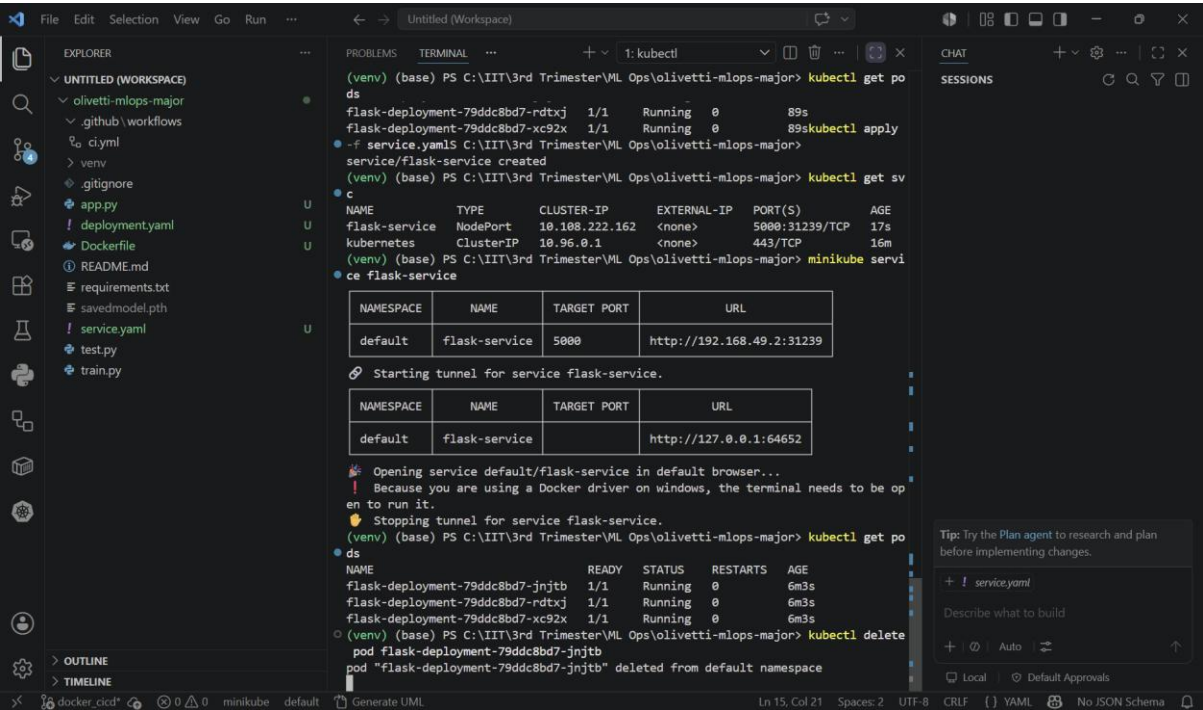
kubectl get pods -w

Result:

- Kubernetes automatically creates a new pod.
- Replica count remains 3.

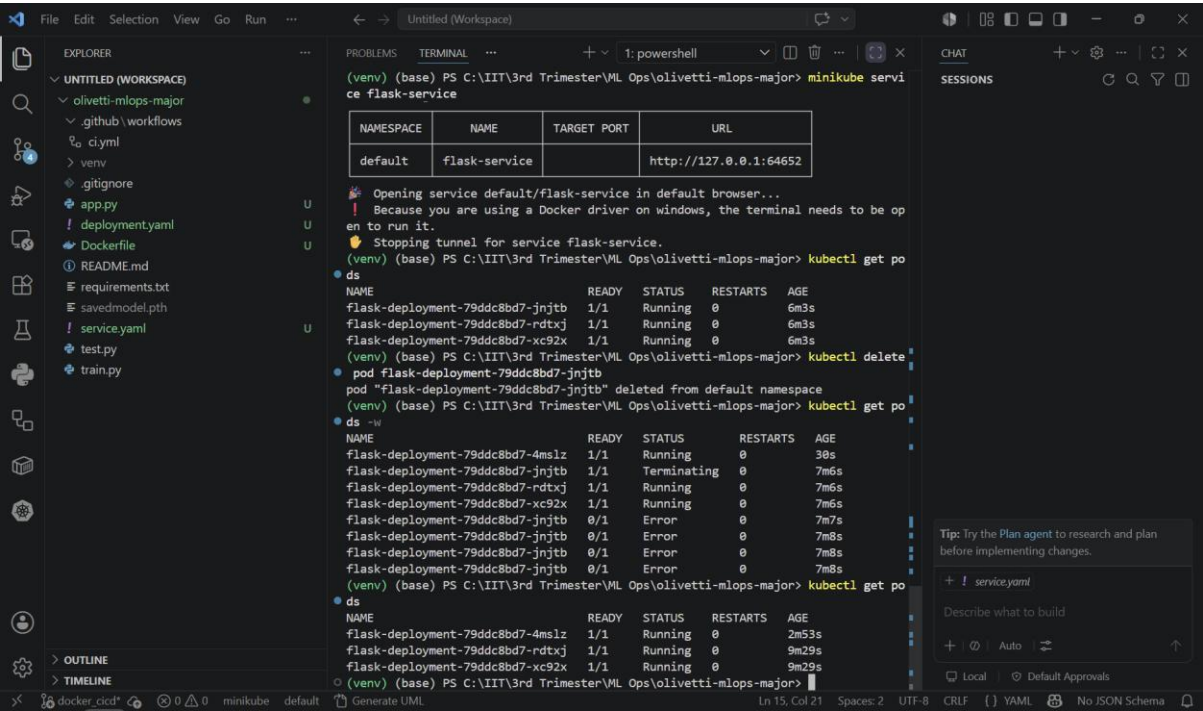
Screenshot

pod deletion



Screenshot

recreated pod



17. Results

Component	Status
Model Training	Successful
Model Testing	Successful
GitHub Actions	Successful
Docker Build	Successful
Docker Hub Push	Successful
Flask Deployment	Successful
Kubernetes Deployment	Successful
Replica Recovery	Successful

18. Challenges Faced

1. Configuring GitHub Actions.
 2. Managing Docker image dependencies.
 3. Connecting Flask with trained model.
 4. Configuring Kubernetes deployment.
 5. Troubleshooting NodePort access.
-

19. Conclusion

A complete MLOps pipeline was successfully implemented using Scikit-Learn, Flask, Docker, GitHub Actions, Docker Hub, and Kubernetes. The workflow automated model training, testing, deployment, and scaling while ensuring high availability through Kubernetes replica management. This project demonstrates the practical application of modern MLOps principles in real-world machine learning deployment scenarios.

20. References

1. [Scikit-Learn Documentation](#)
2. [PyTorch Documentation](#)
3. [Docker Documentation](#)
4. [GitHub Actions Documentation](#)
5. [Kubernetes Documentation](#)